

Assignment 2: Policy Gradient

Andrew ID: mlin4

Collaborators: Claude (used for scripting experiments in parallel)

NOTE: Please do NOT change the sizes of the answer blocks or plots.

5 Small-Scale Experiments (7.5 Pts)

5.1 Experiment 1 (Cartpole) – [4.5 points total]

5.1.1 Configurations

Q5.1.1

```
python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 1500 \
-dsa --exp_name q1_sb_no_rtg_dsa

python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 1500 \
-rtg -dsa --exp_name q1_sb_rtg_dsa

python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 1500 \
-rtg --exp_name q1_sb_rtg_na

python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 6000 \
-dsa --exp_name q1_lb_no_rtg_dsa

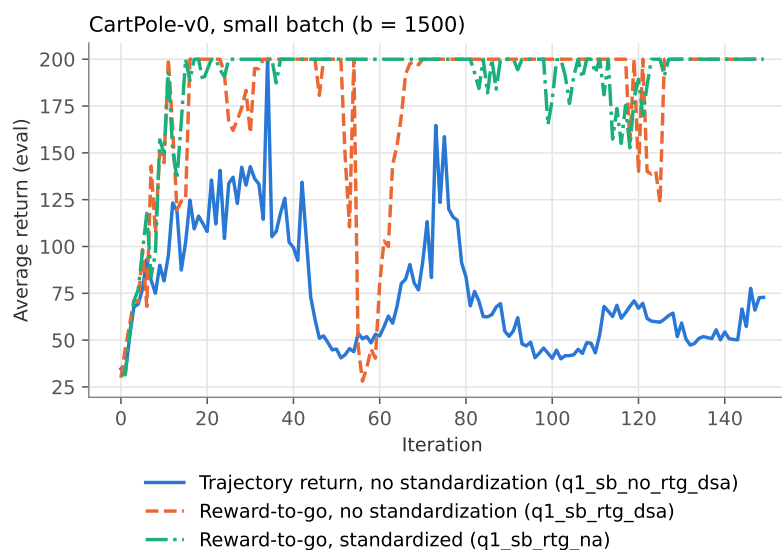
python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 6000 \
-rtg -dsa --exp_name q1_lb_rtg_dsa

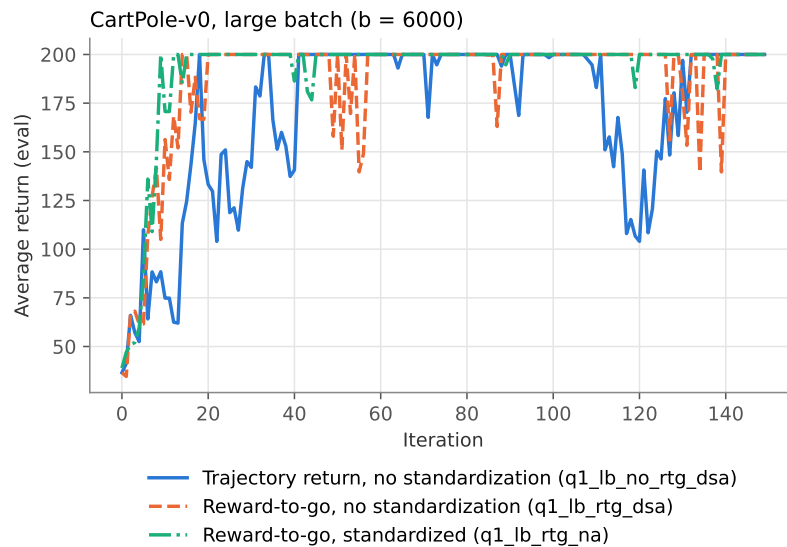
python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 6000 \
-rtg --exp_name q1_lb_rtg_na
```

5.1.2 Plots

5.1.2.1 Small batch – [1 points]

Q5.1.2.1



5.1.2.2 Large batch – [0.5 points]**Q5.1.2.2****5.1.3 Analysis****5.1.3.1 Value estimator – [1 points]****Q5.1.3.1**

Both graphs suggest that reward-to-go performs significantly better than the trajectory-centric method.

5.1.3.2 Advantage standardization – [1 points]**Q5.1.3.2**

Advantage standardization seems to offer a modest performance improvement. Comparing the two reward-to-go runs, the run that uses advantage standardization reaches a return of 200 faster and has less severe dips in reward after capping out.

5.1.3.3 Batch size – [1 points]

Q5.1.3.3

Increased batch size seems to offer benefits both for the trajectory-based and the reward-to-go methods. The trajectory-based method manages to make it to a reward of 200 after increasing the batch size and the reward-to-go methods have less severe dips in reward after capping out in the larger batch experiments.

5.2 Experiment 2 (InvertedPendulum) – [3 points total]

5.2.1 Configurations – [1 points]

Q5.2.1

```
JOBS=${JOBS:-20}
BATCHES=(100 300 500 1000 3000 5000)
LRS=(0.005 0.01 0.02 0.03 0.05)

mkdir -p logs_q2

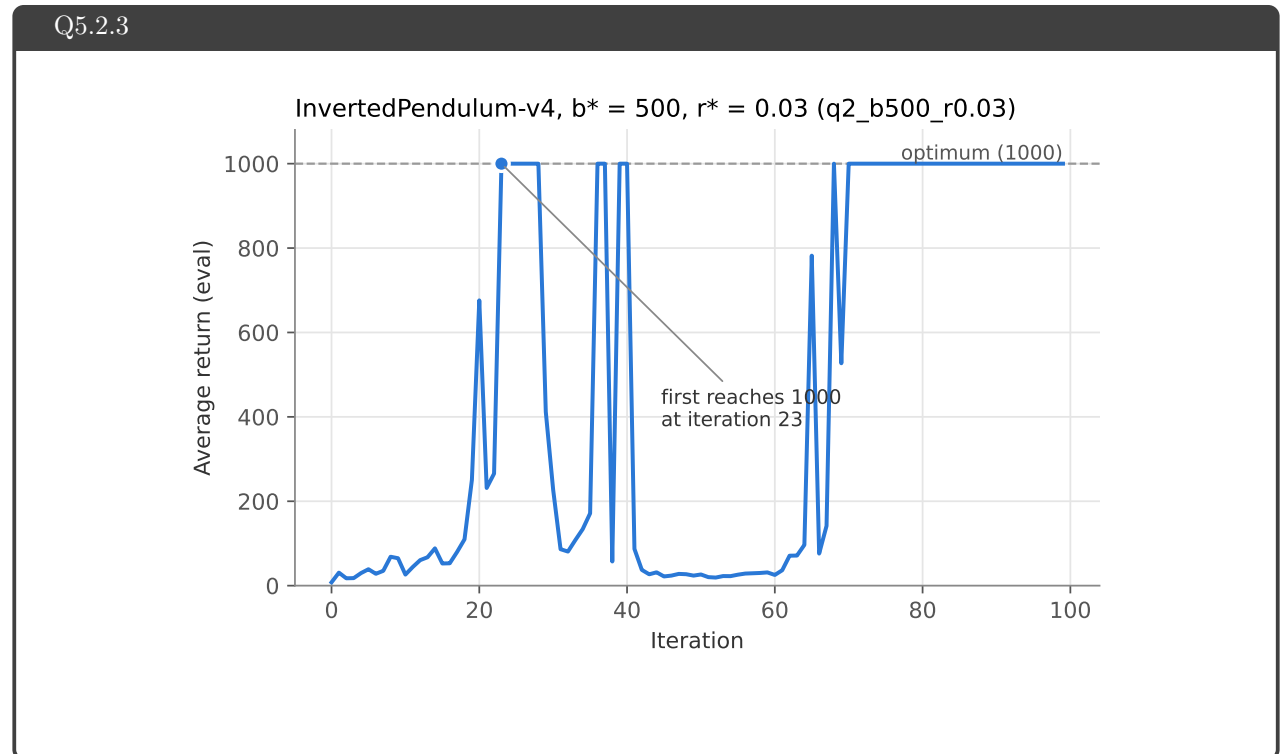
for b in "${BATCHES[@]}; do
  for r in "${LRS[@]}; do
    echo "$b $r"
  done
done | xargs -P "$JOBS" -n 2 sh -c '
b=$0; r=$1
OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 python rob831/scripts/run_hw2.py --env_name InvertedPendulum-v4 \
  --ep_len 1000 --discount 0.92 -n 100 -l 2 -s 64 -b "$b" -lr "$r" -rtg -ngpu \
  --exp_name "q2_b${b}_r${r}" > "logs_q2/q2_b${b}_r${r}.log" 2>&1
echo "finished b=$b r=$r (exit $?)"'
```

5.2.2 smallest b^* and largest r^* (same run) – [1 points]

Q5.2.2

$b^* = 500$ and $r^* = 0.03$

5.2.3 Plot – [1 points]



7 More Complex Experiments (4.5 Pts)

7.1 Experiment 3 (LunarLander) – [1 points total]

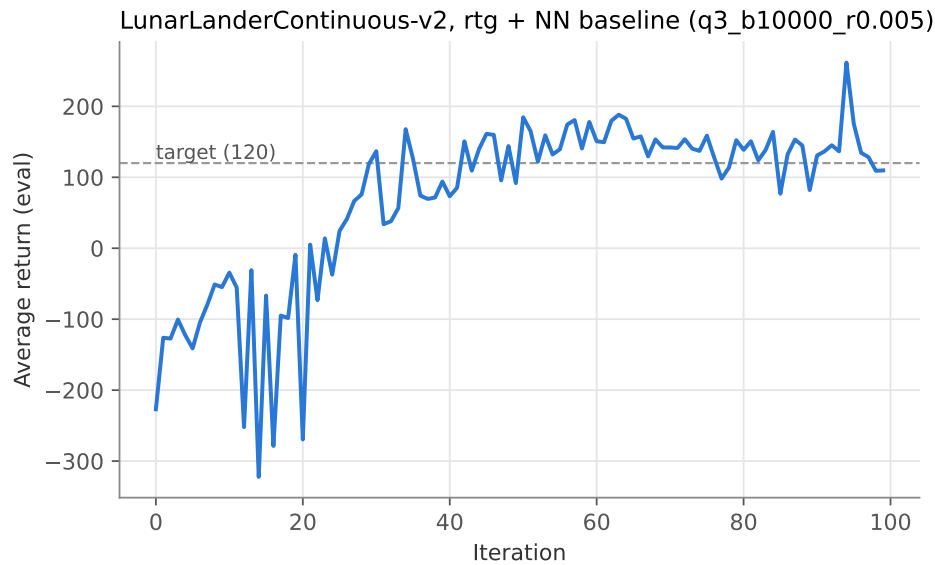
7.1.1 Configurations

Q7.1.1

```
python rob831/scripts/run_hw2.py \  
  --env_name LunarLanderContinuous-v2 --ep_len 1000 \  
  --discount 0.99 -n 100 -l 2 -s 64 -b 10000 -lr 0.005 \  
  --reward_to_go --nn_baseline --exp_name q3_b10000_r0.005 \  
  --seed 5
```

7.1.2 Plot – [1 points]

Q7.1.2



7.2 Experiment 4 (HalfCheetah) – [3.5 points]

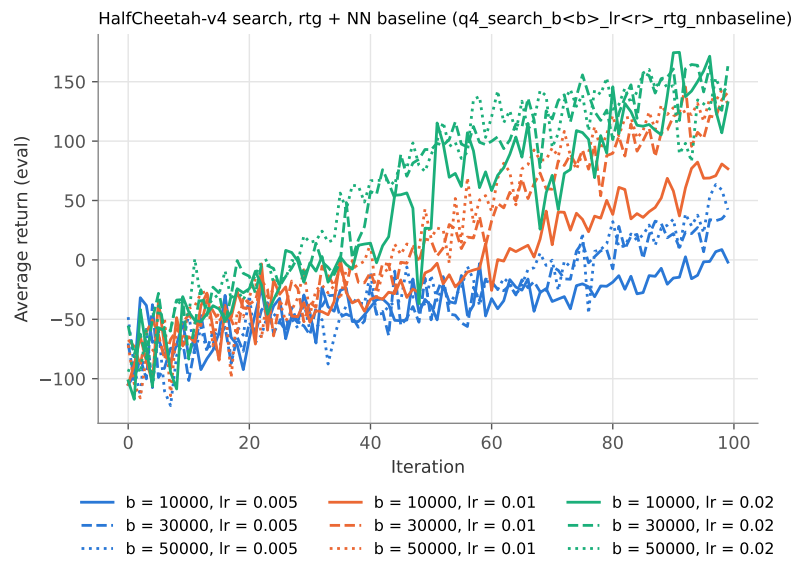
7.2.1 Configurations

Q7.2.1

```
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 --discount 0.95 -n 100 -l 2 -s 32 -b 10000 -lr
↪ 0.005 -rtg --nn_baseline -ngpu --exp_name q4_search_b10000_lr0.005_rtg_nnbaseline
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 --discount 0.95 -n 100 -l 2 -s 32 -b 10000 -lr
↪ 0.01 -rtg --nn_baseline -ngpu --exp_name q4_search_b10000_lr0.01_rtg_nnbaseline
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 --discount 0.95 -n 100 -l 2 -s 32 -b 10000 -lr
↪ 0.02 -rtg --nn_baseline -ngpu --exp_name q4_search_b10000_lr0.02_rtg_nnbaseline
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 --discount 0.95 -n 100 -l 2 -s 32 -b 30000 -lr
↪ 0.005 -rtg --nn_baseline -ngpu --exp_name q4_search_b30000_lr0.005_rtg_nnbaseline
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 --discount 0.95 -n 100 -l 2 -s 32 -b 30000 -lr
↪ 0.01 -rtg --nn_baseline -ngpu --exp_name q4_search_b30000_lr0.01_rtg_nnbaseline
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 --discount 0.95 -n 100 -l 2 -s 32 -b 30000 -lr
↪ 0.02 -rtg --nn_baseline -ngpu --exp_name q4_search_b30000_lr0.02_rtg_nnbaseline
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 --discount 0.95 -n 100 -l 2 -s 32 -b 50000 -lr
↪ 0.005 -rtg --nn_baseline -ngpu --exp_name q4_search_b50000_lr0.005_rtg_nnbaseline
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 --discount 0.95 -n 100 -l 2 -s 32 -b 50000 -lr
↪ 0.01 -rtg --nn_baseline -ngpu --exp_name q4_search_b50000_lr0.01_rtg_nnbaseline
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 --discount 0.95 -n 100 -l 2 -s 32 -b 50000 -lr
↪ 0.02 -rtg --nn_baseline -ngpu --exp_name q4_search_b50000_lr0.02_rtg_nnbaseline
```

7.2.2 Plot – [1 points]

Q7.2.2

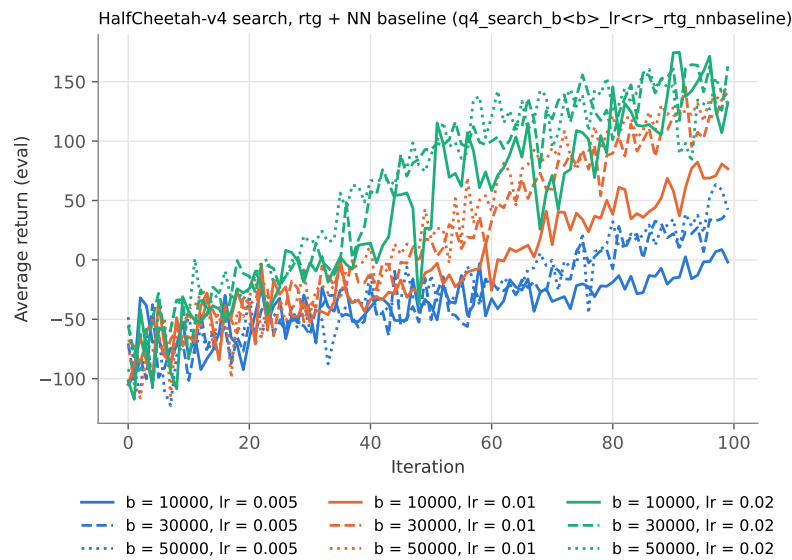
**7.2.3 Optimal b^* and r^* – [0.5 points]**

Q7.2.3

 $b^* = 30000$ and $r^* = 0.02$

7.2.4 Plot – [0.5 points]

Q7.2.4

7.2.5 Describe how b^* and r^* affect task performance – [0.5 points]

Q7.2.5

Increasing learning rate leads to a marked increase in performance when holding batch size steady for any set learning rate. Increasing the batch size doesn't seem to have a strong impact; while $b^* = 10000$ lags behind for most cases, there's often only a small difference between $b^* = 30000$ and $b^* = 50000$.

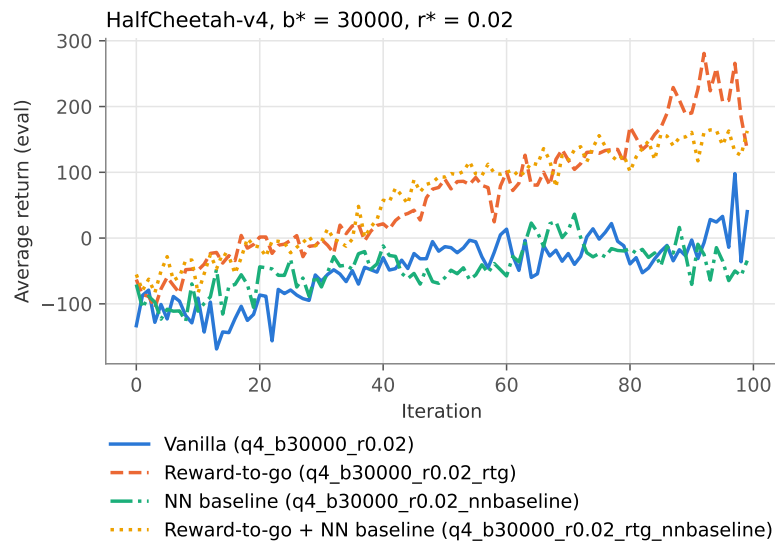
7.2.6 Configurations with optimal b^* and r^* – [0.5 points]

Q7.2.6

```
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 --discount 0.95 -n 100 -l 2 -s 32 -b 30000 -lr
↪ 0.02 -ngpu --exp_name q4_b30000_r0.02
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 --discount 0.95 -n 100 -l 2 -s 32 -b 30000 -lr
↪ 0.02 -rtg -ngpu --exp_name q4_b30000_r0.02_rtg
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 --discount 0.95 -n 100 -l 2 -s 32 -b 30000 -lr
↪ 0.02 --nn_baseline -ngpu --exp_name q4_b30000_r0.02_nnbaseline
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 --discount 0.95 -n 100 -l 2 -s 32 -b 30000 -lr
↪ 0.02 -rtg --nn_baseline -ngpu --exp_name q4_b30000_r0.02_rtg_nnbaseline
```

7.2.7 Plot for four runs with optimal b^* and r^* – [0.5 points]

Q7.2.7



8 Implementing Generalized Advantage Estimation (3 Pts)

8.1 Experiment 5 (Hopper) – [3 points]

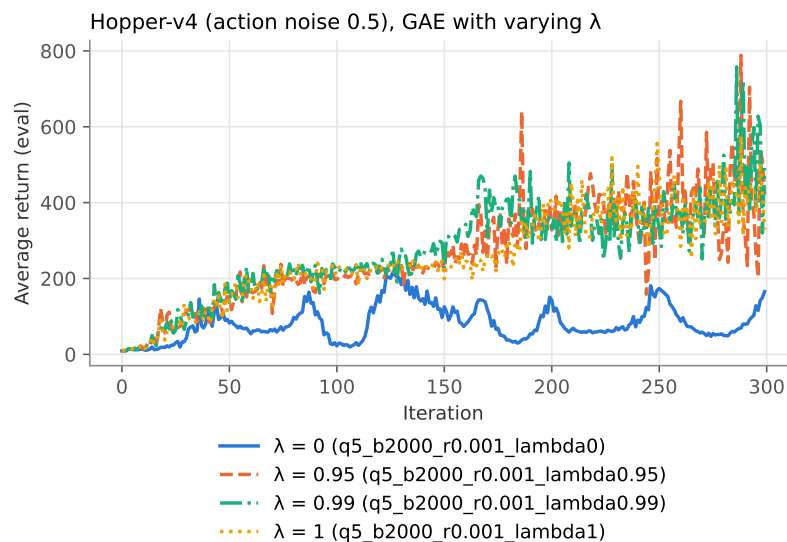
8.1.1 Configurations

Q8.1.1

```
python rob831/scripts/run_hw2.py --env_name Hopper-v4 --ep_len 1000 --discount 0.99 -n 300 -l 2 -s 32 -b 2000 -lr
→ 0.001 --reward_to_go --nn_baseline --action_noise_std 0.5 --gae_lambda 0 -ngpu --exp_name q5_b2000_r0.001_lambda0
python rob831/scripts/run_hw2.py --env_name Hopper-v4 --ep_len 1000 --discount 0.99 -n 300 -l 2 -s 32 -b 2000 -lr
→ 0.001 --reward_to_go --nn_baseline --action_noise_std 0.5 --gae_lambda 0.95 -ngpu --exp_name
→ q5_b2000_r0.001_lambda0.95
python rob831/scripts/run_hw2.py --env_name Hopper-v4 --ep_len 1000 --discount 0.99 -n 300 -l 2 -s 32 -b 2000 -lr
→ 0.001 --reward_to_go --nn_baseline --action_noise_std 0.5 --gae_lambda 0.99 -ngpu --exp_name
→ q5_b2000_r0.001_lambda0.99
python rob831/scripts/run_hw2.py --env_name Hopper-v4 --ep_len 1000 --discount 0.99 -n 300 -l 2 -s 32 -b 2000 -lr
→ 0.001 --reward_to_go --nn_baseline --action_noise_std 0.5 --gae_lambda 1 -ngpu --exp_name q5_b2000_r0.001_lambda1
```

8.1.2 Plot – [1 points]

Q8.1.2



8.1.3 Describe how λ affects task performance – [1.5 points]

Q8.1.3

$\lambda = 0$ leads to much lower performance relative to the other runs that we did. There is no notable performance difference between the three other settings of λ that we tested. However, it is interesting to observe the bias-variance tradeoff that is discussed in the GAE paper: setting a λ near 1 leads to much spikier lines than the $\lambda = 0$ run, which is in line with the consequence of high variance when we have a high λ .

- 8.1.4 Find one robotics paper from the last 5 years that uses policy gradients or an actor-critic method as part of its methodology. In a short paragraph (4–5 sentences): (i) cite the paper, (ii) briefly describe the robot task and how reinforcement learning is used, and (iii) identify one technique from this homework (e.g., reward-to-go, a value-function baseline, or GAE) that is related to the method used in the paper. A few sentences is enough; this is not meant to be an extensive literature review – [0.5 points]**

Q8.1.4

Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning (Rudin et al.) is a paper from 2021 that was published in CoRL. The objective of the paper was to develop a quadruped robot to follow paths over rough terrain (e.g. slopes, stairs). The approach that they used was completely RL-based and train in simulation, where large quantities of "dogs" were trained alongside each other, before Sim2Real transfer and experiments in the real world. They utilize GAE in their approach and use a GAE discount factor of 0.95, which is in line with the near-one values of λ that we tested in our experiments.